Практична робота №6

Розробка власного прикладного програмного інтерфейсу (API)

Тривалість: 90 хв

Формат: комп'ютерний практикум

Мета роботи

Спроєктувати й реалізувати простий прикладний програмний інтерфейс (API) на PHP без використання фреймворків: єдина точка входу з маршрутизацією за URL-параметрами та HTTP-методом, узгоджений формат JSON-відповіді, коректні коди статусу HTTP для різних сценаріїв (успіх, помилка клієнта, відсутній ресурс).

Необхідні знання й інструменти

Знання: виконані практикуми №1–5 (PDO/MySQL, AJAX і формат JSON); базове розуміння HTTP-методів (GET, POST) і кодів статусу (див. лекцію №8 «Прикладний програмний інтерфейс»).

Інструменти: редактор коду; локальний вебсервер із PHP; клієнт для ручного тестування API - браузер (для GET), а також `curl` у терміналі або застосунок Postman/Insomnia (для POST і перевірки заголовків/кодів статусу).

Короткий теоретичний вступ

AJAX-ендпоінт з практикуму №5 писався «під себе» - для власного фронтенду тієї самої сторінки. API - це той самий за суттю механізм (JSON у відповідь на HTTP-запит), але спроєктований як стабільний, документований контракт: ним може скористатися будь-який клієнт (мобільний застосунок, інший сайт, скрипт), а не лише ваш `script.js`.

Єдина точка входу - усі запити до API йдуть на один файл (`api.php`), який визначає, що саме запитується, і викликає відповідний код. Це найпростіший спосіб маршрутизації без фреймворка чи окремого класу-роутера (докладніший підхід із «чистими» URL і класом `Router` - тема практикумів гілки поглибленого вивчення REST API).

```
<?php
header('Content-Type: application/json');
$method   = $_SERVER['REQUEST_METHOD'];
$resource = $_GET['resource'] ?? null;
$id       = $_GET['id'] ?? null;

if ($resource === 'books' && $method === 'GET' && $id === null) {
```

```
    listBooks($pdo);
} elseif ($resource === 'books' && $method === 'GET') {
    getBook($pdo, (int) $id);
} elseif ($resource === 'books' && $method === 'POST') {
    createBook($pdo);
} else {
    http_response_code(404);
    echo json_encode(['success' => false, 'error' => 'Невідомий ресурс або метод']);
}
```

Коди статусу HTTP встановлюються функцією `http_response_code()`. Найпотрібніші для цього практикуму: `200 OK` (успішний GET), `201 Created` (успішно створено новий запис), `400 Bad Request` (клієнт надіслав некоректні дані), `404 Not Found` (ресурс або запис не знайдено), `405 Method Not Allowed` (метод не підтримується для цього ресурсу).

Узгоджений формат відповіді - усі відповіді API мають однакову структуру, незалежно від того, успіх це чи помилка, наприклад: `{"success": true, "data": [...]}` або `{"success": false, "error": "..."}`. Це полегшує роботу будь-якому клієнту API — він завжди знає, де шукати результат чи опис помилки.

Хід роботи

Структура завдання спільна для всіх варіантів - відрізняється лише таблиця бази даних, предметна область і доменна дія (див. розділ «20 варіантів завдання»). Виконуйте кроки послідовно.

Крок 1. Визначити контракт API свого домену.

Опишіть (коментарем на початку `api.php` або окремим текстовим файлом `API.md`) перелік підтримуваних запитів: метод, ресурс, параметри, приклад відповіді - за зразком із розділу «20 варіантів завдання».

Крок 2. Створити єдину точку входу.

У файлі `api.php` встановіть `header('Content-Type: application/json')` одразу на початку, отримайте `$_SERVER['REQUEST_METHOD']` і `$_GET['resource']`.

Крок 3. Реалізувати маршрутизацію.

За допомогою `if/elseif` або `match` (PHP 8) визначте, яка комбінація метод+ресурс прийшла, і викличте відповідну функцію-обробник; для невідомого ресурсу поверніть `http_response_code(404)` і JSON з описом помилки.

Крок 4. Реалізувати GET - список і одиничний запис.

`GET /api.php?resource=<ресурс>` повертає весь список (`200`); `GET /api.php?resource=<ресурс>&id=<N>` повертає один запис за `id` або `404`, якщо такого `id` немає в таблиці.

Крок 5. Реалізувати POST - створення запису.

`POST /api.php?resource=<ресурс>` приймає дані (`$_POST` або JSON-тіло через `php://input`), перевіряє обов'язкові поля (`400`, якщо чогось бракує), виконує `INSERT` через підготовлений запит і повертає `201` з даними створеного запису.

Крок 6. Реалізувати доменну дію, окрему від звичайного CRUD.

Додайте `POST`-дію, специфічну для вашого домену (параметр `action=<...>` у запиті) - див. розділ «20 варіантів завдання»; вона так само повертає JSON і коректний код статусу.

Крок 7. Обробити некоректні запити.

Метод, що не підтримується для вибраного ресурсу, має повертати `405 Method Not Allowed`; параметри, що не пройшли перевірку - `400 Bad Request` з описом, яке саме поле некоректне.

Крок 8. Перевірити API вручну.

Перевірте всі маршрути браузером (`GET`) та `curl -i` або Postman (`POST`), звертаючи увагу на код статусу у відповіді (`curl -i` показує заголовки й код) і структуру JSON-тіла.

20 варіантів завдання

Наскрізні варіанти - той самий номер, що й у попередніх практикумах, відповідає одному домену протягом усього курсу. Виберіть свій варіант і виконуйте завдання для нього.

№ / Домен	Зміст варіанта
1. Бібліотека (каталог книг, видача/повернення)	**Ресурс:** `resource=books` (`id, title, author, year, isbn, is_available`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля перевіряються, `400` при відсутності); з `action=checkout` - видати книгу (позначити `is_available = 0`).
2. Інтернет-магазин (каталог товарів, кошик)	**Ресурс:** `resource=products` (`id, name, price, sku, stock`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля перевіряються, `400` при відсутності); з `action=restock` - поповнити залишок (`stock`) на вказану кількість.
3. Блог/CMS для статей	**Ресурс:** `resource=articles` (`id, title, content, author, published_at`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля перевіряються, `400` при відсутності); з `action=publish` - опублікувати статтю (встановити `published_at = NOW()`).
4. Фітнес-трекер (тренування, вправи)	**Ресурс:** `resource=workouts` (`id, type, duration_min, calories_burned, workout_date`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля

	перевіряються, `400` при відсутності); з `action=stats` - повернути сумарні калорії (`SUM(calories_burned)`) за поточний фільтр.
5. Кінотека (фільми, рецензії)	**Ресурс:** `resource=movies` (`id, title, director, year, genre, rating`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля перевіряються, `400` при відсутності); з `action=rate` - додати оцінку і повернути новий середній `rating`.
6. Кулінарна книга (рецепти, інгредієнти)	**Ресурс:** `resource=recipes` (`id, title, ingredients, cook_time_min`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля перевіряються, `400` при відсутності); з `action=search` - пошук рецептів, у яких `cook_time_min` не перевищує вказане значення.
7. Трекер завдань (to-do list, проєкти)	**Ресурс:** `resource=tasks` (`id, title, due_date, priority, done`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля перевіряються, `400` при відсутності); з `action=complete` - позначити завдання виконаним (`done = 1`).
8. Система бронювання (готель/зали)	**Ресурс:** `resource=rooms` (`id, number, capacity, price_per_night`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля перевіряються, `400` при відсутності); з `action=book` - забронювати кімнату на вказані дати.
9. Форум/дошка обговорень	**Ресурс:** `resource=topics` (`id, title, author, created_at, is_pinned`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля перевіряються, `400` при відсутності); з `action=pin` - закріпити тему (`is_pinned = 1`).
10. Каталог вакансій (job board)	**Ресурс:** `resource=vacancies` (`id, title, company, salary_from, salary_to`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля перевіряються, `400` при відсутності); з `action=stats` - повернути середню зарплату (`AVG`) за поточний фільтр.
11. Система обліку студентів/оцінок	**Ресурс:** `resource=students` (`id, name, group_name, average_grade`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля перевіряються, `400` при відсутності); з `action=top` - повернути студента з найвищим середнім балом у поточній групі.
12. Онлайн-опитування/анкети	**Ресурс:** `resource=questions` (`id, text, type`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля

	перевіряються, `400` при відсутності); з `action=stats` - повернути кількість питань кожного типу (`GROUP BY type`).
13. Трекер особистих витрат (фінанси)	**Ресурс:** `resource=transactions` (`id, amount, category, transaction_date`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля перевіряються, `400` при відсутності); з `action=balance` - повернути поточний баланс (`SUM(amount)`).
14. Каталог музики/плейлисти	**Ресурс:** `resource=tracks` (`id, title, artist, duration_sec`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля перевіряються, `400` при відсутності); з `action=stats` - повернути сумарну тривалість (`SUM(duration_sec)`) за поточний фільтр.
15. Система бронювання квитків (кіно/події)	**Ресурс:** `resource=events` (`id, title, event_date, venue, seats_left`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля перевіряються, `400` при відсутності); з `action=buy` - купити квиток (зменшити `seats_left` на 1).
16. Дошка оголошень	**Ресурс:** `resource=listings` (`id, title, price, category`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля перевіряються, `400` при відсутності); з `action=stats` - повернути найдешевше оголошення в поточній категорії.
17. Трекер звичок (habit tracker)	**Ресурс:** `resource=habits` (`id, name, target_per_week, current_streak`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля перевіряються, `400` при відсутності); з `action=checkin` - зареєструвати виконання сьогодні (`current_streak += 1`).
18. Каталог догляду за рослинами (садівництво)	**Ресурс:** `resource=plants` (`id, name, watering_interval_days, last_watered_at`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля перевіряються, `400` при відсутності); з `action=water` - позначити рослину политою (`last_watered_at = CURDATE()`).
19. Система тікетів служби підтримки (helpdesk)	**Ресурс:** `resource=tickets` (`id, subject, priority, status`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля перевіряються, `400` при відсутності); з `action=close` - закрити тікет (`status = 'closed'`).
20. Спортивна ліга (турнірна таблиця, матчі, команди)	**Ресурс:** `resource=teams` (`id, name, city, points`). **GET:** без `id` - список; з `id` - один запис або `404`. **POST:** без `action` - створення (обов'язкові поля

	перевіряються, `400` при відсутності); з `action=addPoints` - нарахувати команді очки за результат матчу.